

Migrating the Trading Desk backend from WSGI (Bottle) to ASGI (FastAPI)

Repository	trading-desk, branch hw-5.5-asgi-migration
Commits in stage order	criteria 237fd81 → benchmark a3ed7f0, results 339128b → decision 3ef4b1c → risks 9f5ae79 → alternative plan dd9f522, 5dfb3cf and its results
Development	Apple M3, 8 cores, 16 GB, Docker Desktop 29.4.3
Benchmark host	cloud VM, Intel Xeon 2.1 GHz, 4 vCPU, 15 GB, Ubuntu 24.04 (section 3)
Versions	Python 3.14.7, Bottle 0.13.4, SQLAlchemy 2.0.52, psycopg 3.3.4, PostgreSQL 18.6

1. Summary

NO-GO: the services stay on Bottle. At the concurrency this system actually has, FastAPI and Bottle on threaded workers serve the same when waiting on another service (181 vs 182 req/s at $c = 10$); at $c = 50$ FastAPI runs out of CPU at 451 req/s where threads reach 723; and the migration would cost 59 hours against a 40-hour limit. Instead, `desk-runtime` now serves all six services with gunicorn threads instead of the hand-rolled `wsgiref` server: +19–24 % throughput on the real books code at $c \geq 10$, and no lost requests at $c = 200$ where the old server dropped up to 8.4 %.

2. System inventory (Stage 0)

Six Bottle services in Docker Compose, one PostgreSQL database, one React frontend. Each service is one process in one container, served by a small threaded server from `desk-runtime` (`wsgiref` with one thread per connection, `service_runtime.py`), using a synchronous database driver (SQLAlchemy + psycopg). There are no automated tests.

Every service is **I/O-bound**: handlers wait on the database or on another service, never on computation. Endpoint details are in appendix A.

The load is small. One browser polls a few endpoints every 2–10 s and holds three SSE streams; monitoring calls every `/health` every 5 s; pricing and blotter each hold one internal stream. No service has more than about five requests in flight at once.

2.1 Services

market-data-service · 8001

Does	Polls 7 external providers, stores quotes and curves, streams updates to clients
Endpoints	19 — quotes, curves, watchlist, FX, search, providers, <code>/snapshot</code> , <code>/stream</code>
Depends on	PostgreSQL, 7 external APIs
Load	Handlers read the DB. Provider calls — the slow part — run in background threads
Runs as	1 process, polling threads, SSE subscribers in memory

pricing-service · 8002

Does	Revalues active trades from the market stream, streams valuations to clients
Endpoints	7 — <code>/valuations</code> , <code>/book-risk</code> , <code>POST /price</code> , <code>POST /scenario</code> , <code>/valuation-stream</code>
Depends on	PostgreSQL, market-data-service (stream + snapshot)
Load	Reads serve memory. The work is the incoming stream and a periodic DB refresh
Runs as	1 process, stream consumer thread, prices and valuations in memory

monitoring-service · 8003

Does	Checks the health of every service and the DB, tails the log files
Endpoints	5 — <code>/status</code> , <code>/audits</code> , <code>/logs</code> , <code>/logs/stream</code>
Depends on	PostgreSQL, all five other services (<code>/health</code> every 5 s), log files
Load	Its own health polling is most of its work
Runs as	1 process, six polling threads, log buffers in memory

books-service · 8004

Does	Stores trading books; refuses to close a book that still has open trades
Endpoints	6 — <code>GET/POST /books</code> , <code>GET/PUT/DELETE /books/<id></code>
Depends on	PostgreSQL only
Load	One or two DB queries per request, nothing else
Runs as	1 process, no background threads, nothing in memory

trade-action-service · 8008

Does	Validates trade orders, acknowledges them, executes them from a queue
Endpoints	6 — <code>POST /trade-actions</code> (+ <code>/batch</code> , <code>/close-all</code>), <code>/queue/status</code> , <code>/instruments/term-schemas</code>
Depends on	PostgreSQL only
Load	Validation queries the DB in the request; the write happens on a worker thread
Runs as	1 process, one worker thread, queue in memory

blotter-service · 8006

Does	Shows trades with their latest valuations, book summaries, audit history
Endpoints	7 — <code>/trades</code> , <code>/trades/overview</code> , <code>/trades/<id></code> (+ valuations, audit), <code>/books/summary</code>
Depends on	PostgreSQL, pricing-service (stream + snapshot)
Load	Several DB queries per request, joined with a valuation cache
Runs as	1 process, stream consumer thread, trades and valuations in memory

Five services keep live data only in their own memory: pricing holds the latest valuations, trade-action holds its order queue, and so on. Running two copies of such a service would split that data between them — a request to one copy would not see what the other holds. So these five can only

ever run as one copy each. books-service keeps nothing in memory; several copies of it would behave exactly like one. That matters for the benchmark, which the PDF runs with several worker processes: for books-service that setup is real, for the others it could not exist.

2.2 Benchmark sample: books-service

A benchmark builds the same thing twice, in Bottle and in FastAPI. Everything except the framework must be identical, or the comparison measures something else. books-service is the easiest service to hold identical:

- **No background threads** — nothing runs beside the handlers to add noise.
- **Nothing in memory** — no cache or queue to fake before measuring.
- **No other services** — only the database is needed to run it.

What remains is what every service here does: take a request, run one or two synchronous database queries, return JSON.

The others fail one of these tests. blotter and pricing serve caches that only exist while their streams run; market-data's `/snapshot` pulls in most of the domain library; monitoring's `/logs` needs the log collector.

2.3 Scenarios

The PDF prescribes a small sample app, built in both frameworks, with a few endpoints. Three are artificial on purpose — each isolates one property — and one uses the real books code.

#	Endpoint	Does	Answers
S1	GET <code>/health</code>	Returns tiny JSON, no I/O	Bare cost of the framework and server
S2	GET <code>/io</code>	Calls a stub service that waits 50 ms	Waiting for another service — where ASGI should win
S3	GET <code>/cpu</code>	20 000 rounds of SHA-256	Pure computation — where ASGI cannot help. A control
S4	GET <code>/books</code>	Real books-service code: one query, 20 books as JSON	What migration does to <i>this</i> system. No gain expected; shows nothing breaks

Each scenario runs at $c = 1, 10, 50$ and 200 against three variants: Bottle on sync workers, Bottle on threaded workers and FastAPI. The variants, the thresholds and the reason the threaded Bottle variant is included are in `docs/decision_criteria.md`.

Held SSE connections are not measured. One browser and two internal consumers hold five streams in total — a load no server here notices.

3. Benchmark method

The criteria were committed on their own (`237fd81`) before any benchmark code existed. Everything below is in `benchmark/` and runs with one command, `benchmark/run_benchmark.sh`; `benchmark/README.md` says how to reproduce it.

Sample. One small app written twice, with the same four endpoints (2.3). `sample_wsgi` is Bottle; its `/books` is the real books-service app merged in unchanged. `sample_asgi` is FastAPI; its `/books` calls the same repository function and the same JSON serializer. S2 calls a stub that sleeps 50 ms: Bottle

with `requests`, FastAPI with one shared `httpx.AsyncClient` created in the lifespan — the PDF's appendix A, unchanged.

Variants. Handler types follow the rule the migration would follow: `async def` only where the I/O does not block.

Variant	Server	S1	S2	S3	S4
A	gunicorn, 1 sync worker	def	def	def	def
A' (A-threads)	gunicorn, 1 worker × 40 threads	def	def	def	def
B	uvicorn, 1 worker, plain install (asyncio loop, h11 parser)	async def	async def + httpx	def (thread pool)	def (thread pool)

Grid. 4 scenarios × c = 1, 10, 50, 200 × 3 variants × 3 runs = 144 measurements. Each is 10 s of warm-up, discarded, then 30 s with `hey -z 30s -c <c> -t 10`. The three variants run one after another at every point, so a slow moment on the machine hits all of them. A monitor samples CPU and RSS of the server once a second.

Fairness rules from the PDF (5.1).

Pitfall	Here
Bottle's development server	never used — gunicorn for A and A'
debug, reload, console access log	none on either side
no warm-up, short or single runs	10 s warm-up, 30 s runs, 3 repetitions, median and min-max
client on the same cores as the server	server pinned to core 0; client, stub and database on cores 1-3
blocking calls inside <code>async def</code>	only S1 and S2 are <code>async</code> ; S3 and S4 are <code>def</code>
different process counts	one worker everywhere
default client timeout hiding failures	<code>-t 10</code> ; errors reported next to latency, > 1 % loses the point
a stub that is itself the bottleneck	checked alone at c = 200 before the grid

Two changes from the PDF's example, both found in the smoke test.

1. *One worker, not four.* Every service runs as one process today, so N = 1 is the representative setting. And with `--workers` above 1, uvicorn 0.53 creates its socket without the TCP protocol number, so asyncio never turns on `TCP_NODELAY`. Each keep-alive request then waits about 40 ms for a delayed ACK: `/health` at c = 1 had p95 0.3 ms with one worker and 44 ms with two. Measuring that would compare a socket option, not WSGI and ASGI.
2. *One stub process, not two* — the same defect added 40 ms to every call from B's keep-alive `httpx` client, and to none from A's `requests`, which opens a new connection each time.

Hardware and versions. A cloud VM, not the development laptop: Intel Xeon 2.1 GHz, 4 vCPU, 15 GB, Ubuntu 24.04. Python 3.14.7, Bottle 0.13.4, gunicorn 26.2.0, FastAPI 0.141.1, uvicorn 0.53.0, httpx 0.28.1, SQLAlchemy 2.0.52, psycopy 3.3.4, hey 0.1.5. PostgreSQL 16.13 on the same VM. Full list: `benchmark/requirements.txt`.

Limitations.

- One core per server makes the numbers small. Ratios between variants matter here, not absolute throughput; on the laptop they will be larger.
- The load generator, the stub and PostgreSQL share three cores. At a few thousand requests per second they compete with each other, though never with the server.

- A shared cloud VM has noisy neighbours. The min-max bars show how much that mattered.
- PostgreSQL 16 instead of the project's 18.6. S4 reads 20 rows with one simple query, which behaves the same on both.

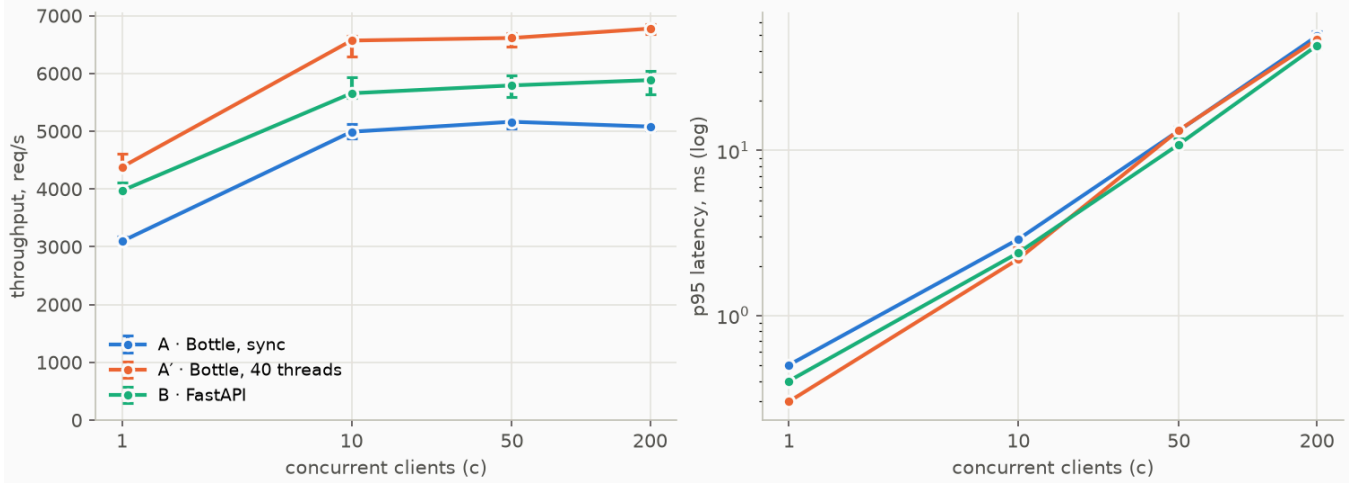
4. Results

144 measurements on 2026-09-22, 22:26-00:10 UTC. Median of three runs, min-max in brackets. The only errors in the whole grid are A's timeouts in S2 at $c = 200$. Full tables with p50, p99, CPU and RSS: [benchmark/results/summary.md](#); raw hey output: [benchmark/results/](#).

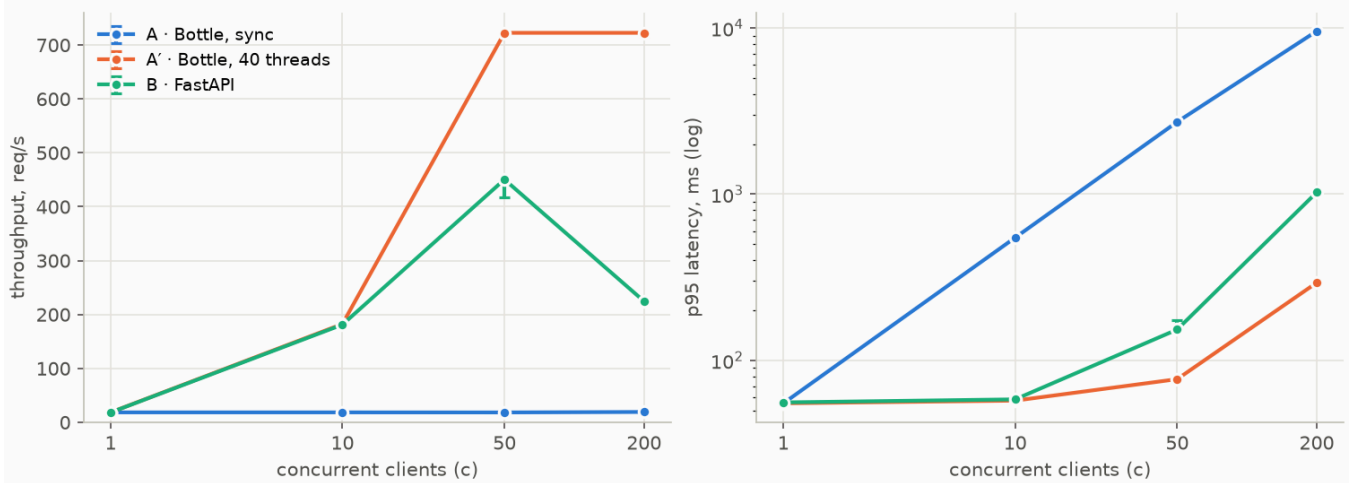
Scenario	c	A · Bottle sync	A' · Bottle 40 threads	B · FastAPI
S1 /health	10	4990 req/s · p95 2.9 ms	6569 req/s · p95 2.2 ms	5656 req/s · p95 2.4 ms
S2 /io (50 ms wait)	10	18 req/s · p95 548 ms	182 req/s · p95 57 ms	181 req/s · p95 58 ms
S2 /io	50	18 req/s · p95 2724 ms	723 req/s · p95 77 ms	451 req/s · p95 153 ms
S2 /io	200	19 req/s · 78 % timeouts	723 req/s · p95 293 ms	225 req/s · p95 1026 ms
S3 /cpu	10	164 req/s · p95 72 ms	165 req/s · p95 108 ms	160 req/s · p95 120 ms
S4 /books	10	541 req/s · p95 23 ms	668 req/s · p95 22 ms	706 req/s · p95 23 ms
S4 /books	50	548 req/s · p95 108 ms	710 req/s · p95 227 ms	732 req/s · p95 98 ms

Memory barely differs: 103 MB (A), 113 MB (A'), 118 MB (B) at the highest load.

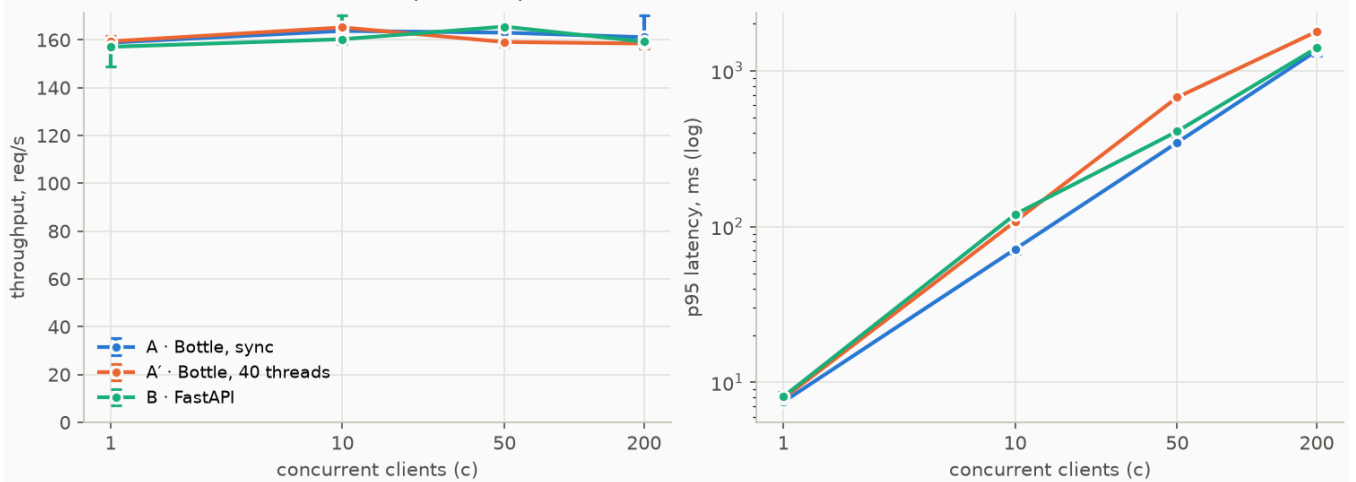
S1 /health — framework overhead · median of 3 runs, bars = min-max

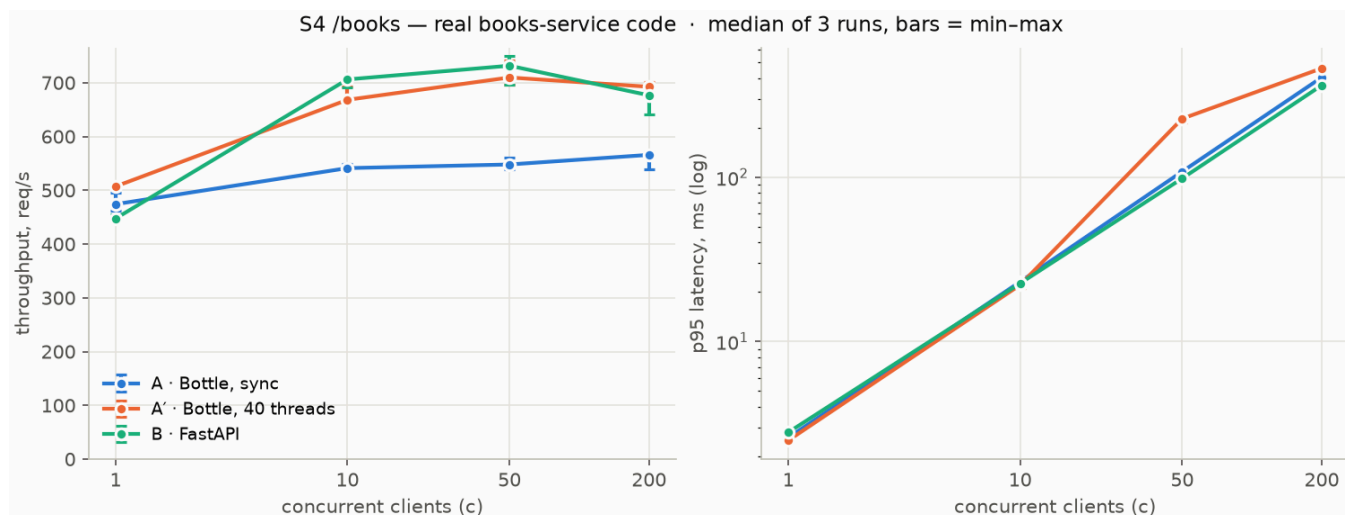


S2 /io — waits 50 ms on another service · median of 3 runs, bars = min-max



S3 /cpu — computation · median of 3 runs, bars = min-max





Gates (computed by `benchmark/analyze.py` from the rules in `docs/decision_criteria.md`):

Gate	Point	Result	Verdict
1	S1, c = 10	B −14 % throughput against A'	fail
1	S3, c = 10	no difference	pass
1	S4, c = 10	B +6 % against A'	pass
2	S2, c = 50	B against A': p95 +99 %, throughput −38 %	fail
3	cost ≤ 40 h	Stage 2	see 6
4	S2, c = 10	no difference	fail

5. Interpretation

Waiting (S2) is where WSGI and ASGI really differ, and the numbers follow simple arithmetic. A sync worker holds one request for its whole 54 ms, so it serves $1 / 0.054 = 18$ requests a second at any concurrency; everyone else queues, and p95 grows as $c \times 54$ ms: 548 ms at $c = 10$, 2.7 s at $c = 50$, timeouts at $c = 200$. Threads remove that limit up to their number: A' matches B at $c = 10$ (182 vs 181 req/s = $10 / 0.055$ s), and at $c = 50$ it stops at 40 threads / 0.055 s ≈ 727 req/s — measured 723. That is the textbook WSGI problem, and one server flag solves it for this range.

The event loop waits for free, but each request still costs CPU. B has no thread limit, yet at $c = 50$ it did 451 req/s with its core at 89 %: about 2.0 ms of CPU per `/io` request, against 1.3 ms for A'. On one core the CPU ran out before the unlimited waiting could pay off. The extra cost sits in the pure-Python layers of B — the `httpx` client, the `h11` parser and the `asyncio` loop of a plain `uvicorn` install. Above $c = 50$ B lost throughput (225 req/s at $c = 200$) while A' held 723. The likely cause is `httpx`'s default pool of 100 connections: the other requests queue inside the client, and its bookkeeping eats the core. Stage 4 repeats S1–S4 with `uvicorn[standard]` (`uvloop`, `httptools`) to check how much of this is the installation rather than ASGI (8.4: it doubles S1, and leaves S2 where it is).

Computation (S3) is the control, and it behaves. Every variant does ~ 160 req/s — one core, ~ 6 ms of hashing per request. No server helps with that. Tails differ: A serves requests one after another (p95 72 ms at $c = 10$); in A' and B threads share the GIL and every request is stretched (108 and 120 ms).

The real code (S4) shows neither a gain nor a loss. B is 6 % ahead of A' at $c = 10$, equal at $c = 50$ and 200 , and keeps a shorter tail under saturation (p95 98 ms against A's 227 ms at $c = 50$). With a synchronous database driver FastAPI runs the handler in a thread pool, exactly like A' — so there is nothing for async to win, which is what the plan predicted.

The bare framework (S1) is fastest on threaded gunicorn. B beats sync Bottle by 13 % but trails A' by 14 %, failing Gate 1 at that point.

Uncertainty. Spreads are mostly 1–5 % of the median (the widest: B in S2 at $c = 50$, 418–454 req/s). Every difference the gates rely on is many times the spread, except where the table says *no difference*. The verdict would not change with a 10 % error in either direction.

What the results do not say. They are one core per server on a shared cloud VM, so the absolute numbers are small; the ratios are the result. They do not cover an asynchronous database driver, several processes, or FastAPI on uvloop (Stage 4 adds the last one). And they measure one sample service — the other five would add streams and background threads, not new kinds of work.

Surprises. The `TCP_NODELAY` defect in multi-worker uvicorn (section 3) would have added 40 ms to every B request had the smoke test not caught it. And in the one scenario B was expected to win, it lost to threaded Bottle above $c = 10$ — by running out of CPU, not threads.

6. Criteria and decision (ADR)

ADR-001: Keep Bottle (WSGI); do not migrate to FastAPI (ASGI)

Date: 2026-09-23. **Status:** accepted.

Context. Six Bottle services, one process each, synchronous database driver, no tests (section 2). The load is one browser, health checks and two internal streams — at most about five requests in flight per service. The question is the one teams ask regularly: should we move to FastAPI?

Decision criteria. Four gates, committed on their own in `237fd81` before any benchmark code existed (`docs/decision_criteria.md`). GO needs all four.

Cost estimate (Gate 3). From the inventory, per service. Assumptions: contract tests 0.25 h per endpoint (none exist, and GO requires them); porting 0.5 h per endpoint (Pydantic models in and out, the old status codes and `{"error": ...}` format); 2 h per served SSE stream; 1.5 h per service with background threads or in-memory state (lifespan, ordering).

Service	Endpoints	Tests	Port	Streams	Threads, state	Hours
market-data	19	4.75	9.5	2	1.5	17.75
pricing	7	1.75	3.5	2	1.5	8.75
monitoring	5	1.25	2.5	2	1.5	7.25
books	6	1.5	3	—	—	4.5
trade-action	6	1.5	3	—	1.5	6
blotter	7	1.75	3.5	—	1.5	6.75
shared: desk-runtime 4 h, Docker and README 2 h, after-benchmark 2 h						8
Total	50	12.5	25	6	7.5	59

Even at half the porting time (0.25 h per endpoint) the total is 46.5 h, above the 40 h limit.

Data.

Gate	Measured	Verdict
1 — nothing gets worse	S3 no difference; S4 B +6 %; S1 B –14 % against threaded Bottle	fail (S1)
2 — gain where it should	S2, c = 50: B 451 req/s, p95 153 ms; threaded Bottle 723 req/s, p95 77 ms	fail
3 — cost $\leq$ 40 h	59 h (46.5 h if porting is twice as fast)	fail
4 — the system needs it	S2, c = 10: 181 vs 182 req/s, p95 58 vs 57 ms — no difference	fail

Spreads are 1–5 %; no verdict sits near its threshold. The top three risks (section 7) are blocking calls on the event loop, silent contract changes and the absence of tests.

Options considered.

Option	For	Against	Cost
1. Full migration to FastAPI	Pydantic validation instead of hand-written checks; OpenAPI; ready for WebSocket and many streams	no measured gain at this load; every handler stays <code>def</code> until the database driver changes; 12 risks; <code>asyncio</code> to learn	59 h
2. Stay on WSGI, fix the server	threaded gunicorn was the best variant in S1 and S2; production server instead of <code>wsgiref</code> ; one file changes	no automatic validation or OpenAPI; a fixed pool of 40 threads	2–3 h, measured in Stage 4
3. Partial or deferred: books-service only, or wait for a trigger	keeps the FastAPI option warm	two runtimes to maintain (risk 10); still no gain	4.5 h for books

Decision. NO-GO. All four gates fail, and the decisive one is Gate 4: at the concurrency this system actually has, FastAPI and threaded Bottle measure the same (181 vs 182 req/s). Where `async` should shine — many clients waiting at once — threads already cover up to 40 in flight, and FastAPI ran out of CPU before it could pass them. Pydantic and OpenAPI are real benefits but do not buy back 59 hours of work with no measured gain.

Consequences.

- Code: `desk-runtime` moves from `wsgiref` to gunicorn with threads (Stage 4B). Services keep Bottle and synchronous handlers; nothing about their endpoints changes.
- Operation: a bounded pool of 40 threads per service. Every open SSE stream holds one, so a few dozen browser tabs would starve the rest — a revisit condition below.
- Team: no `asyncio` to learn; hand-written request validation stays.
- The FastAPI sample stays in `benchmark/sample_asgi/` as a proof of concept for a future revision.

Revisit when any of these happens:

- a service regularly has more than about 40 requests in flight — the thread limit where threaded Bottle's p95 starts to grow in S2;
- more than about 20 SSE clients per service, or WebSocket;
- an asynchronous database driver is adopted — only then can FastAPI handlers stop using the thread pool;
- a service needs more than one process.

7. Risk analysis

Risks of migrating to FastAPI, specific to this system. P = probability, I = impact, L / M / H = low / medium / high. The three most serious come first.

#	Risk	Category	P	I	Mitigation	Warning signal
1	A blocking call inside <code>async def</code> stalls every request in the process. All 54 database sessions (<code>session_scope()</code>) are synchronous, and so are the provider clients and the stream consumers	technical	H	H	<code>async def</code> only for non-blocking I/O, <code>def</code> everywhere the database is touched; ruff <code>ASYNC</code> rules; load test per service	p99 jumps under load while CPU stays low; section 8.3 measures how much
2	Silent contract changes: 422 instead of 400, <code>{"detail"}</code> instead of <code>{"error"}</code> , 307 redirects for trailing slashes, string-to-int coercion. The frontend reads <code>body.error</code> from failures	technical	H	M	contract tests recorded on Bottle first; custom exception handlers returning <code>{"error": ...}</code> with the old codes; <code>redirect_slashes=False</code>	the frontend shows empty or <code>undefined</code> error messages
3	No automated tests today, so regressions go unnoticed. Writing them is 12.5 h of the 59 h estimate	process	H	H	contract tests before the first migrated service — worth doing even without the migration	an endpoint changes behaviour and nobody sees it until the browser does
4	SSE streams read <code>queue.Queue</code> from a sync generator. Under FastAPI each open stream holds one of the 40 thread-pool slots that <code>def</code> handlers also need	technical	M	H	rewrite the three streams as async generators over an async queue, or cap stream count	<code>/health</code> slows down when a few browser tabs are open
5	<code>uvicorn --workers</code> above 1 leaves <code>TCP_NODELAY</code> off: +40 ms on every keep-alive request (measured in Stage 1)	operational	M	M	one worker (services cannot run more anyway), or gunicorn with <code>UvicornWorker</code>	p50 near 40 ms on a trivial endpoint
6	Background threads and in-memory state (pricing cache, trade queue, stream subscribers) must start in the lifespan, in the same process as the handlers	operational	M	H	start threads in the lifespan; one worker; test restart and ordering (blotter bootstraps before serving)	empty blotter or duplicate polling after a restart
7	Thread pool and database pool disagree: 40 <code>def</code> threads share a pool of 5 + 10 connections, so bursts wait up to 30 s for a connection	performance	M	M	size <code>pool_size</code> to the thread count, or cap threads	<code>QueuePool limit ... timed out</code> in logs; flat throughput with idle CPU
8	<code>httpx.AsyncClient</code> costs more CPU per call than <code>requests</code> on one core; in S2 B ran out of CPU at 451 req/s where threaded Bottle did 723	performance	M	M	one shared client from the lifespan, explicit limits, measure before and after	CPU at 100 % with throughput below the Bottle baseline
9	The estimate is low: market-data alone has 19 endpoints and seven provider adapters	process	M	M	books-service first as the pilot; extrapolate its real time; stop and return to NO-GO if the pilot exceeds 1.5× its estimate	the pilot overruns

#	Risk	Category	P	I	Mitigation	Warning signal
10	Half-migrated system under time pressure: some services on Bottle, some on FastAPI, two runtimes to maintain	organizational	M	M	service by service with a fixed stop rule; shared desk-runtime pieces migrated first	after half the time, fewer than half the services are done
11	One developer new to <code>asyncio</code> : missing <code>await</code> , shared state across tasks, no timeouts	organizational	M	M	checklist in review; explicit timeouts on every network call; <code>asyncio</code> debug mode in development	"coroutine was never awaited" warnings; requests that hang
12	Dependency drift between services (Pydantic, httpx, anyio)	technical	L	L	already one <code>requirements.txt</code> for the whole repository	different versions in <code>pip freeze</code> of two images

8. Alternative plan (Stage 4B)

8.1 Why NO-GO, and when to look again

The data that decided it: at $c = 10$ — twice today's peak — FastAPI and threaded Bottle measure the same in S2 (181 vs 182 req/s), and the migration costs 59 h against a 40 h limit. The gain async promises needs many clients waiting at once; this system has about five requests in flight per service. Revisit when a service regularly holds more than ~ 40 requests in flight, when more than ~ 20 SSE clients or WebSocket are needed, when an asynchronous database driver is adopted, or when a service needs a second process (section 6).

8.2 The plan

#	Item	Cost	Expected effect	Status
1	Serve every service with gunicorn threads (1 worker $\times$ 40) instead of <code>wsgiref</code>	2–3 h	the best WSGI variant of Stage 1 in production: HTTP/1.1 keep-alive, a bounded thread pool, a listen backlog of 2048 instead of 5, a graceful stop, no per-request log line on stderr	done , measured in 8.3
2	Contract tests for the 50 endpoints, recorded on today's code	12.5 h	catch regressions in any later change, including a future migration	next
3	Size the database pool to the thread count (<code>pool_size</code>)	0.5 h	no 30 s waits for a connection when 40 threads burst at once (risk 7)	when load grows
4	FastAPI with an async driver for market-data only, the one service with many waiting clients (streams)	~ 20 h	the only place ASGI could pay off	only on a revisit trigger

8.3 Item 1: what changed and what it measured

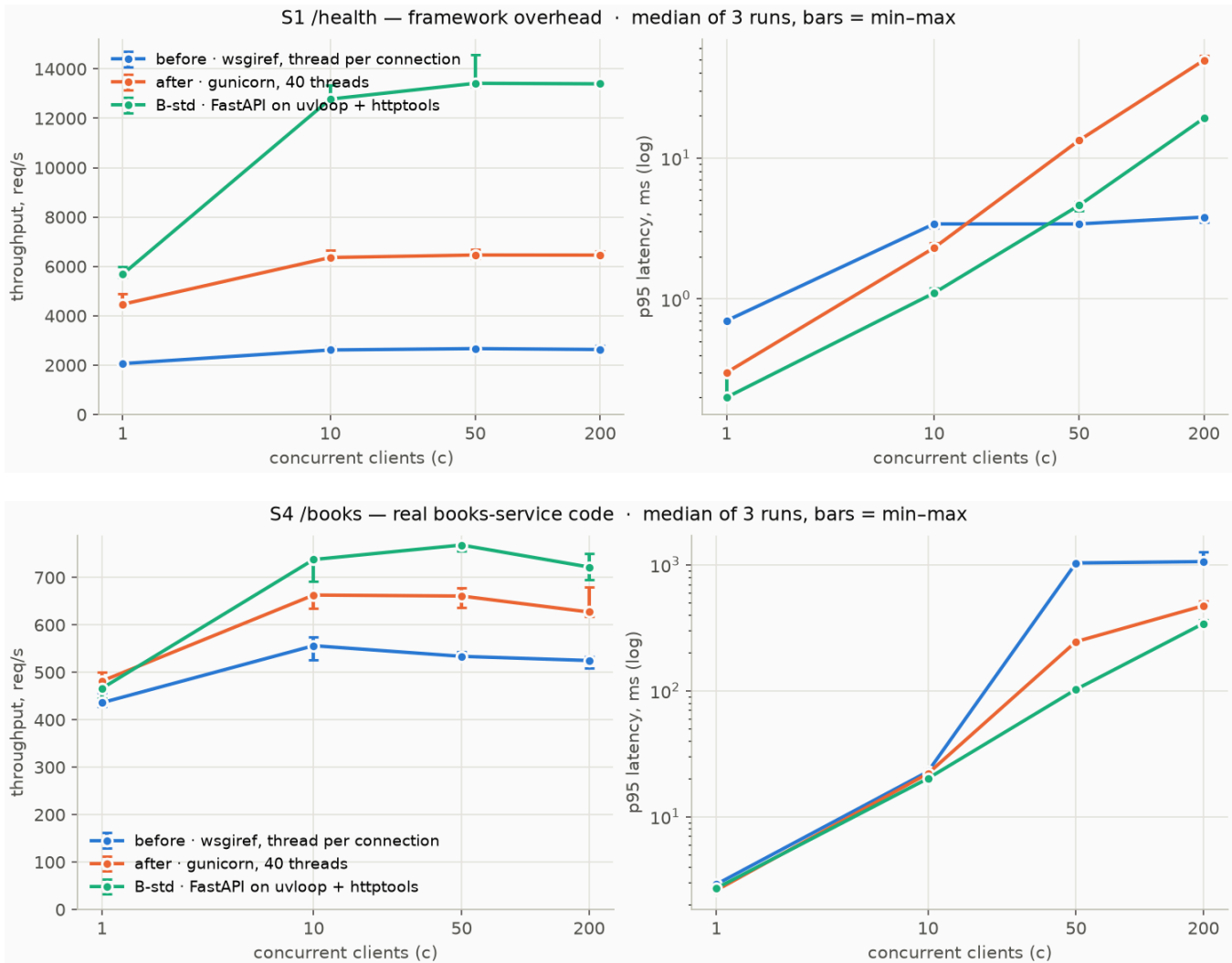
`desk-runtime`'s `service_runtime.py` is the only runtime file that changed (dd9f522): `ThreadedServer` (`wsgiref` + `ThreadingMixIn`) became `ServiceServer`, a gunicorn application with one worker and `HTTP_THREADS` threads. Startup hooks and background threads now start inside the worker, next to the handlers that read their state; market-data's curve pruning moved into a startup hook so no database connection is opened before the fork. The API contract is unchanged; so is every service's code apart from one line in market-data.

Checked on all six services running locally: book create/read/update/delete with the old status codes and `{"error": ...}` bodies for 400, 404, 405 and 409; the three SSE streams; pricing reconnecting when market-data restarts; keep-alive; a stop within 7 s with a stream held open. No tracebacks, no worker timeouts.

Before/after benchmark — the same sample, scenarios and grid as Stage 1 (`results-4b/`). *before* is the sample on the old server, *after* on `ServiceServer`.

Scenario	c	before · wsgiref	after · gunicorn threads	Change
S1 <code>/health</code>	10	2614 req/s	6360 req/s	+143 %
S1 <code>/health</code>	200	2633 req/s · p99 1039 ms · 0.4 % errors	6458 req/s · p99 90 ms · no errors	+145 %
S2 <code>/io</code>	10	186 req/s	185 req/s	no difference
S2 <code>/io</code>	50	604 req/s · p99 1083 ms	728 req/s · p99 88 ms	+20 %
S2 <code>/io</code>	200	625 req/s · p95 1096 ms · 0.7 % errors	723 req/s · p95 297 ms · no errors	+16 %, p95 –73 %
S3 <code>/cpu</code>	200	164 req/s · 8.4 % errors	156 req/s · no errors	no difference in throughput
S4 <code>/books</code>	10	556 req/s	663 req/s	+19 %
S4 <code>/books</code>	50	534 req/s · p95 1033 ms	661 req/s · p95 245 ms	+24 %, p95 –76 %
S4 <code>/books</code>	200	525 req/s · 1.6 % errors	627 req/s · no errors	+19 %

Full tables and charts: `benchmark/results-4b/summary.md` and `benchmark/results-4b/charts/`.



What the old server was doing wrong shows in two numbers. It speaks HTTP/1.0, so every request opens a new connection; and `socketserver` listens with a queue of **five**. Above a handful of clients, connection attempts overflow that queue and the client retries after 1 s, then 3 s — hence a p99 of about a second from $c = 50$ and timeouts at $c = 200$. Its p95 at $c \geq 50$ in S1 even looks *better* than the new server's (3.4 against 13.3 ms), and that is the trap: the waiting happens before `accept()`, where percentiles of answered requests do not see it. Little's law gives it away — 200 clients at 2633 req/s means 76 ms per request on average, while the median is 2 ms.

The price is memory: 153 MB RSS instead of 90 MB, because gunicorn runs a master and a worker and both hold the application (RSS counts the pages they share twice). *after* also reproduces Stage 1's threaded variant, measured two hours earlier with the same settings: within 5 % in S1–S3 and within 10 % in S4, where the database adds its own variation.

Behaviour that differs, deliberately. Connections stay open between requests (HTTP/1.1). A stop waits up to 5 s for requests in flight instead of cutting them. The per-request line that `wsgiref` printed to stderr is gone; the application's own logs, which monitoring collects, are unchanged.

Cost. Estimated 2–3 h; actual about 1.5 h including the six-service check. The before/after run took 1 h 45 min of machine time.

8.4 Two checks on Stage 1

FastAPI on uvloop and httptools (B-std, what uvicorn[standard] installs), measured in the same run as *after*:

Point	after · threaded Bottle	B-std · FastAPI on uvloop	Stage 1's B · plain uvicorn
S1, c = 10	6360 req/s	12 777 req/s (+101 %)	5656 req/s
S2, c = 10	185 req/s	184 req/s (no difference)	181 req/s
S2, c = 50	728 req/s	481 req/s (−34 %)	451 req/s
S4, c = 10	663 req/s	738 req/s (+11 %)	706 req/s
S4, c = 50	661 req/s · p95 245 ms	768 req/s · p95 102 ms (+16 %)	732 req/s · p95 98 ms

The C event loop and parser double FastAPI's bare throughput, so Gate 1's failure in S1 came from the plain install and would pass with uvicorn[standard]. They do not change S2: FastAPI still loses to threads at c = 50 and still ties at c = 10. The cost there is the async HTTP client per request, not the server. Gates 2, 3 and 4 fail either way; the decision stands.

One blocking call on the event loop (blocking_demo.sh, PDF appendix A): /health on FastAPI, one client, while ten others run the S3 computation.

Beside /health	p50	p95	/health req/s
nothing	0.2 ms	0.2 ms	5755
/cpu declared def (thread pool)	1.4 ms	45.5 ms	80
the same work declared async def	57.4 ms	63.6 ms	17

With def the event loop stays free and the median barely moves; the tail still grows, because the hashing threads hold the GIL. With async def every /health waits behind the whole queue of computations — about ten of them at 6 ms each. That is risk 1 in numbers, and why every database handler in a FastAPI version of this system would have to stay def.

8.5 Proof of concept

benchmark/sample_asgi/ stays in the repository: the four endpoints on FastAPI, with the lifespan-managed httpx client and def handlers for blocking work. It is the starting point if a revisit condition is ever met.

9. Conclusions and lessons

Answers to the assignment's questions.

1. *What problem would the migration solve, and does it exist here?* Many clients waiting on I/O at once. It does not exist here: about five requests in flight per service, and at c = 10 FastAPI and threaded Bottle measure the same (section 2, Gate 4).
2. *What does each scenario measure, and why is it representative?* S1 framework overhead, S2 waiting on another service (pricing and blotter call other services), S3 computation as a control, S4 the real books-service code — one or two synchronous queries per request, which is what every service does (section 2.3).
3. *How large is the uncertainty?* Spreads of 1–5 % of the median; every difference the decision rests on is many times larger, and the rest is reported as *no difference* (section 5).

4. *Which criterion decided it, and what would change it?* Gate 4. More than ~40 requests in flight per service, many SSE clients or WebSocket, or an async database driver would (section 6).
5. *The three most serious risks?* Blocking calls on the event loop, silent contract changes, no tests (section 7).
6. *(GO) Cost against the estimate* — not applicable.
7. *(NO-GO) What instead, what did it give, when to return?* A production server in `desk-runtime`: gunicorn threads instead of `wsgiref`, which gave +19–24 % throughput on the real books code at $c \geq 10$, +143 % on bare requests, and no lost requests at $c = 200$ where the old server dropped up to 8.4 % (section 8). Return on any revisit condition in section 6.
8. *What did this teach about benchmarks and about async programming?* Below.

About benchmarks.

- Failed requests do not appear in percentiles. The old server looked fine at $c = 200$ by p95 while it was dropping requests on a listen queue of five; sync Bottle "had" a p95 of 9.6 s while 78 % of its requests timed out. Always read errors next to latency.
- The layer under the application can dominate the result: a missing `TCP_NODELAY` added 40 ms to every request, a listen backlog of five lost requests. A three-minute smoke test found the first before it could spoil 144 measurements.
- A good benchmark agrees with arithmetic. $1 / 54 \text{ ms} = 18 \text{ req/s}$ for a sync worker, 40 threads / $55 \text{ ms} = 727 \text{ req/s}$ for the threaded one — measured 18 and 723. When the numbers do not match the model, something in the setup is wrong, or the model is.
- The fair baseline is the cheapest alternative, not the weakest. Against sync Bottle, FastAPI wins S2 by 25×; against threaded Bottle, one flag away, it loses.

About async.

- Async saves threads, not CPU. The event loop waited on any number of requests, but each cost 2 ms of CPU against 1.3 ms for threads, and on one core the CPU ran out first.
- With a synchronous database driver a FastAPI handler is a `def` in a thread pool — the same model as threaded Bottle, with more layers. The framework alone makes nothing asynchronous.
- One blocking call in `async def` stalls everything: `/health` went from 0.2 ms to about 57 ms at the median while ten clients ran a computation on the event loop.

What I would do differently. Put the server the services actually ran on into Stage 1 from the start; it turned out to be the weakest variant, and Stage 4 found that only by measuring it. Write the contract tests before anything else — every path of this assignment needed them. Run the grid once more on the development laptop, to show the ratios hold on other hardware.

10. Appendices

A. Endpoints

All responses are JSON. Errors are `{"error": "..."}` with the HTTP status — also for unknown routes (404), wrong methods (405) and unhandled exceptions (500). SSE endpoints return `text/event-stream`.

market-data-service

Method	Path	Params	Codes	Body
GET	/stream, /stream/<provider>	—	200, 404	SSE: market_tick, curve_tick, market_remove
GET	/snapshot	—	200	{stream_id, event_id, spots, curves}
GET	/curves, /curves/<provider>	raw	200, 404	list of curves
GET	/curves/<provider>/<curve>/<as_of>	raw	200, 400, 404	one curve
POST	/curves/refresh	curve, provider	200, 404	{refreshed, skipped}
GET	/quotes	symbol, asset_class, provider	200	list of quotes
GET	/quotes/<provider>/<symbol>	—	200, 404	one quote
GET	/quotes/<provider>/<symbol>/history	limit 1-200, raw	200, 400, 404	list of snapshots
GET	/watchlist	—	200	list of items
POST	/watchlist	JSON body	201, 400, 404, 409, 422	item
DELETE	/watchlist/<symbol>	provider	200, 404, 409	removal result
GET	/fx/rates	to	200, 400	{to, rates}
GET	/symbols/search	q (≥ 2 chars)	200, 400, 503	{query, results, provider_errors}
GET	/providers, /providers/<name>/health	—	200, 404	provider status
POST	/refresh	symbol, provider	200, 404, 422	tick
GET	/health	—	200	{service, status}

pricing-service

Method	Path	Params	Codes	Body
GET	/valuations	—	200	list of valuations
GET	/valuations/<trade_id>	—	200, 404	one valuation
GET	/book-risk	—	200	list of book-risk records
POST	/price	JSON body	200, 400, 404, 409, 503	priced instrument
POST	/scenario	JSON body	200, 400, 404	scenario result
GET	/valuation-stream	—	200	SSE: valuation_update, book_risk_update
GET	/health	—	200	{service, status, ...}

monitoring-service

Method	Path	Params	Codes	Body
GET	/status	—	200	health of every target
GET	/audits	limit, since, severity, service, event_type, correlation_id, entity_id	200	list of audit rows
GET	/logs	level, since_id, run_id, service, q, limit	200, 400	{lines, meta}
GET	/logs/stream	—	200	SSE: run, log_line
GET	/health	—	200	{service, status}

books-service

Method	Path	Params	Codes	Body
GET	/books	—	200	list of books
GET	/books/<book_id>	—	200, 404	one book
POST	/books	name, expected_asset_class, description	201, 400, 409	created book
PUT	/books/<book_id>	partial body	200, 400, 404, 409	updated book
DELETE	/books/<book_id>	—	200, 404, 409	deactivated book
GET	/health	—	200	{service, status}

trade-action-service

Method	Path	Params	Codes	Body
GET	/instruments/term-schemas	—	200	{instruments, schemas, curves}
POST	/trade-actions	JSON intent	202, 400, 422, 503	acknowledgement
POST	/trade-actions/batch	JSON array	202, 400, 413, 422, 503	{accepted, rejected}
POST	/trade-actions/close-all	optional body	202, 400, 503	acknowledgement
GET	/queue/status	—	200	counters
GET	/health	—	200	{service, status}

blotter-service

Method	Path	Params	Codes	Body
GET	/trades, /trades/overview	limit 1-500, offset, book_id, asset_class, status, symbol	200, 400	list of trades / {trades, books}
GET	/trades/<trade_id>	—	200, 404	{trade, latest_valuation, valuation_history, audit_logs}

Method	Path	Params	Codes	Body
GET	/trades/<trade_id>/valuations, .../audit-logs	—	200, 404	list
GET	/books/summary	—	200	list of book summaries
GET	/health	—	200	{service, status, ...}

B. Versions and commands

Pinned in `benchmark/requirements.txt` (benchmark) and `requirements.txt` (services): Python 3.14.7, Bottle 0.13.4, gunicorn 26.2.0, FastAPI 0.141.1, Starlette 1.6.0, uvicorn 0.53.0 (with uvloop 0.22.1 and httptools 0.8.0 for B-std only), httpx 0.28.1, requests 2.34.2, SQLAlchemy 2.0.52, psycopg 3.3.4; hey 0.1.5; PostgreSQL 16.13.

```
benchmark/run_benchmark.sh # Stage 1
VARIANTS="before after B-std" RESULTS=results-4b benchmark/run_benchmark.sh # Stage 4B
RESULTS=results-4b benchmark/blocking_demo.sh # demo, 8.4
python benchmark/analyze.py [benchmark/results-4b] # tables, charts,
gates
```

C. Full results

`benchmark/results/summary.md` (Stage 1) and `benchmark/results-4b/summary.md` (Stage 4B): every point with median and min-max of throughput and p95, p50, p99, error rate, CPU and RSS. Raw hey output for each of the 288 measurements and the per-second CPU/RSS samples sit next to them.